PRÁCTICA 03

STL en C++

Vector, list, algoritmos y compilación en Visual Studio

Tema	Contenedores y algoritmos de la biblioteca estándar
Lenguaje	C++17
Compilador	MSVC 19.44 · Visual Studio 2022
Modalidad	Individual
Fecha	Octubre de 2026
Entrega	Cuatro programas, solución de Visual Studio y documento

Documento de soluciones con explicación, código fuente y evidencia de ejecución.

Contenido

- 1. Presentación
- 2. Vector y ordenamiento
- 3. Promedio y mayor número
- 4. Lista de tareas
- 5. Estadísticas de números
- 6. Compilación y ejecución en Visual Studio
- 7. Conclusiones
- Anexo A. Configuración CMake

1. Presentación

La práctica tiene como objetivo aplicar los contenedores vector y list, junto con algoritmos de la STL como sort, max_element, min_element, accumulate y count_if. Los programas solicitan datos al usuario, validan la entrada y presentan resultados formateados.

El código no contiene comentarios ni marcas de edición. Cada archivo es un programa independiente y todos se compilan con el estándar C++17. La carpeta incluye una solución de Visual Studio generada con CMake para ejecutar cada ejercicio sin configuración adicional.

Elemento	Detalle
Lenguaje	C++17
Compilador	MSVC 19.44 (Visual Studio 2022)
Solución	build/PracticaSTL.sln
Ejercicio 1	ejercicio1_vector_ordenamiento.cpp
Ejercicio 2	ejercicio2_promedio_mayor.cpp
Ejercicio 3	ejercicio3_lista_tareas.cpp
Reto extra	reto_extra_estadisticas.cpp

Contenedor o algoritmo	Uso en la práctica	Complejidad
vector<int>	Almacenamiento contiguo y recorrido.	Acceso O(1)
sort	Ordenamiento de menor a mayor.	O(n log n)
max_element / min_element	Búsqueda del mayor y del menor.	O(n)
accumulate	Suma de los elementos.	O(n)
list<string>	Inserción y eliminación al inicio.	pop_front O(1)
count_if	Conteo de pares e impares.	O(n)

2. Vector y ordenamiento

Ejercicio 1. Vector y ordenamiento

ENUNCIADO

Crear un vector<int>, pedir 6 números al usuario, mostrar los números ingresados, ordenarlos de menor a mayor y mostrar el resultado.

SOLUCIÓN

Se reserva capacidad para seis elementos y se leen con validación de entrada. Después se recorre el vector con un for basado en rangos, se aplica std::sort y se vuelve a mostrar. std::sort ordena in situ y trabaja con los iteradores begin y end.

CÓDIGO

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

namespace {

constexpr int CANTIDAD_NUMEROS = 6;

int leerEntero(const std::string& mensaje) {
    int valor;
    while (true) {
        std::cout << mensaje;
        if (std::cin >> valor) {
            return valor;
        }
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Entrada invalida. Debe ingresar un numero entero.\n";
    }
}

std::vector<int> leerNumeros(int cantidad) {
    std::vector<int> numeros;
    numeros.reserve(cantidad);
    for (int i = 0; i < cantidad; ++i) {
        numeros.push_back(leerEntero("Numero " + std::to_string(i + 1) + ": "));
    }
    return numeros;
}

void mostrarNumeros(const std::vector<int>& numeros, const std::string& etiqueta) {
    std::cout << etiqueta;
    for (int numero : numeros) {
        std::cout << numero << ' ';
    }
    std::cout << '\n';
}

}

int main() {
    std::cout << "Ordenamiento de numeros con std::vector y std::sort\n\n";
    std::vector<int> numeros = leerNumeros(CANTIDAD_NUMEROS);

    mostrarNumeros(numeros, "\nNumeros ingresados: ");
    std::sort(numeros.begin(), numeros.end());
    mostrarNumeros(numeros, "Numeros ordenados: ");

    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Ordenamiento de numeros con std::vector y std::sort

Numero 1: 5
Numero 2: 3
Numero 3: 8
Numero 4: 1
Numero 5: 9
Numero 6: 2

Numeros ingresados: 5 3 8 1 9 2
Numeros ordenados: 1 2 3 5 8 9
```

La función leerEntero evita que una entrada no numérica deje el flujo en estado de error. Si cin falla, se limpia el estado y se descarta el resto de la línea antes de volver a solicitar el dato.

3. Promedio y mayor número

Ejercicio 2. Promedio y mayor número

ENUNCIADO

Permitir el ingreso de números enteros hasta que el usuario escriba -1, sin guardar el -1. Mostrar todos los números, el mayor y el promedio.

SOLUCIÓN

El centinela -1 controla el ciclo y se descarta antes de guardar. Si el vector queda vacío, el programa informa que no hay datos. El mayor se obtiene con max_element y la suma con accumulate; el promedio se calcula con double para conservar decimales.

CÓDIGO

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <limits>
#include <numeric>
#include <string>
#include <vector>

namespace {

constexpr int CENTINELA = -1;

int leerEntero(const std::string& mensaje) {
    int valor;
    while (true) {
        std::cout << mensaje;
        if (std::cin >> valor) {
            return valor;
        }
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Entrada invalida. Debe ingresar un numero entero.\n";
    }
}

std::vector<int> leerNumeros() {
    std::vector<int> numeros;
    while (true) {
        int valor = leerEntero("Ingrese un numero (-1 para terminar): ");
        if (valor == CENTINELA) {
            break;
        }
        numeros.push_back(valor);
    }
    return numeros;
}
```

```
void mostrarNumeros(const std::vector<int>& numeros) {
    std::cout << "Numeros ingresados: ";
    for (int numero : numeros) {
        std::cout << numero << ' ';
    }
    std::cout << '\n';
}

}

int main() {
    std::cout << "Promedio y mayor numero con std::vector\n\n";
    std::vector<int> numeros = leerNumeros();

    if (numeros.empty()) {
        std::cout << "\nNo se ingresaron numeros para analizar.\n";
        return 0;
    }

    const int mayor = *std::max_element(numeros.begin(), numeros.end());
    const double suma = std::accumulate(numeros.begin(), numeros.end(), 0.0);
    const double promedio = suma / static_cast<double>(numeros.size());

    mostrarNumeros(numeros);
    std::cout << "Cantidad de numeros: " << numeros.size() << '\n';
    std::cout << "Numero mayor: " << mayor << '\n';
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Promedio: " << promedio << '\n';

    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Promedio y mayor numero con std::vector

Ingrese un numero (-1 para terminar): 10
Ingrese un numero (-1 para terminar): 7
Ingrese un numero (-1 para terminar): 4
Ingrese un numero (-1 para terminar): 15
Ingrese un numero (-1 para terminar): 8
Ingrese un numero (-1 para terminar): -1

Numeros ingresados: 10 7 4 15 8
Cantidad de numeros: 5
Numero mayor: 15
Promedio: 8.80
```

Pista STL	Aplicación
vector	Contenedor dinámico que crece con push_back.
push_back	Agrega cada número válido al final.
max_element	Devuelve un iterador al mayor elemento.
accumulate	Suma todos los elementos desde un valor inicial.
for basado en rangos	Recorre el vector para mostrar los datos.

4. Lista de tareas

Ejercicio 3. Lista de tareas

ENUNCIADO

Crear una list<string> vacía, permitir al usuario agregar 5 tareas, mostrar todas las tareas numeradas, eliminar la primera tarea y volver a mostrar la lista.

SOLUCIÓN

Se utiliza getline para leer tareas con espacios y se valida que no estén vacías. La lista se recorre con un for basado en rangos y un contador para la numeración. pop_front elimina la primera tarea en tiempo constante.

CÓDIGO

```
#include <iostream>
#include <list>
#include <string>

namespace {

constexpr int CANTIDAD_TAREAS = 5;

std::string leerTarea(int numero) {
    std::string tarea;
    while (true) {
        std::cout << "Tarea " << numero << ": ";
        std::getline(std::cin, tarea);
        if (!tarea.empty()) {
            return tarea;
        }
        std::cout << "La tarea no puede estar vacia.\n";
    }
}

void mostrarTareas(const std::list<std::string>& tareas, const std::string& titulo) {
    std::cout << '\n' << titulo << '\n';
    int indice = 1;
    for (const std::string& tarea : tareas) {
        std::cout << indice << ". " << tarea << '\n';
        ++indice;
    }
}

}

int main() {
    std::cout << "Lista de tareas con std::list\n\n";
    std::list<std::string> tareas;

    for (int i = 1; i <= CANTIDAD_TAREAS; ++i) {
        tareas.push_back(leerTarea(i));
    }

    mostrarTareas(tareas, "Tareas registradas:");

    if (!tareas.empty()) {
        tareas.pop_front();
    }

    mostrarTareas(tareas, "Lista actualizada despues de eliminar la primera tarea:");

    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Lista de tareas con std::list

Tarea 1: Estudiar Python
Tarea 2: Hacer la tarea
Tarea 3: Comprar materiales
Tarea 4: Leer un libro
Tarea 5: Descansar

Tareas registradas:
1. Estudiar Python
2. Hacer la tarea
3. Comprar materiales
4. Leer un libro
5. Descansar
```

```
Lista actualizada despues de eliminar la primera tarea:
1. Hacer la tarea
2. Comprar materiales
3. Leer un libro
4. Descansar
```

La diferencia clave entre vector y list es el costo de las operaciones. list facilita insertar y eliminar al inicio, pero no permite acceso aleatorio por índice. Para este ejercicio, pop_front expresa directamente la intención de eliminar la primera tarea.

5. Estadísticas de números

Reto extra. Conteo de pares e impares

ENUNCIADO

Modificar el ejercicio 2 para que también muestre cuántos números son pares y cuántos son impares.

SOLUCIÓN

count_if recorre el vector y aplica una lambda que comprueba el residuo módulo 2. Los impares se calculan restando los pares al total. El programa conserva el centinela y las validaciones del ejercicio anterior.

CÓDIGO

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <limits>
#include <numeric>
#include <string>
#include <vector>

namespace {

constexpr int CENTINELA = -1;

int leerEntero(const std::string& mensaje) {
    int valor;
    while (true) {
        std::cout << mensaje;
        if (std::cin >> valor) {
            return valor;
        }
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Entrada invalida. Debe ingresar un numero entero.\n";
    }
}

std::vector<int> leerNumeros() {
    std::vector<int> numeros;
    while (true) {
        int valor = leerEntero("Ingrese un numero (-1 para terminar): ");
        if (valor == CENTINELA) {
            break;
        }
        numeros.push_back(valor);
    }
    return numeros;
}

void mostrarEstadisticas(const std::vector<int>& numeros) {
    const int mayor = *std::max_element(numeros.begin(), numeros.end());
    const int menor = *std::min_element(numeros.begin(), numeros.end());
    const double suma = std::accumulate(numeros.begin(), numeros.end(), 0.0);
    const double promedio = suma / static_cast<double>(numeros.size());
    const auto pares = std::count_if(numeros.begin(), numeros.end(), [](int numero) {
        return numero % 2 == 0;
```

```
    });
    const auto impares = static_cast<long long>(numeros.size()) - pares;

    std::cout << "\nNumeros ingresados: ";
    for (int numero : numeros) {
        std::cout << numero << ' ';
    }

    std::cout << "\nCantidad de numeros: " << numeros.size();
    std::cout << "\nNumero mayor: " << mayor;
    std::cout << "\nNumero menor: " << menor;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "\nPromedio: " << promedio;
    std::cout << "\nNumeros pares: " << pares;
    std::cout << "\nNumeros impares: " << impares << '\n';
}

}

int main() {
    std::cout << "Reto extra: estadisticas de una lista de numeros\n\n";
    std::vector<int> numeros = leerNumeros();

    if (numeros.empty()) {
        std::cout << "\nNo se ingresaron numeros para analizar.\n";
        return 0;
    }

    mostrarEstadisticas(numeros);
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Reto extra: estadisticas de una lista de numeros

Ingrese un numero (-1 para terminar): 3
Ingrese un numero (-1 para terminar): 8
Ingrese un numero (-1 para terminar): 2
Ingrese un numero (-1 para terminar): 7
Ingrese un numero (-1 para terminar): 10
Ingrese un numero (-1 para terminar): -1

Numeros ingresados: 3 8 2 7 10
Cantidad de numeros: 5
Numero mayor: 10
Numero menor: 2
Promedio: 6.00
Numeros pares: 3
Numeros impares: 2
```

6. Compilación y ejecución en Visual Studio

La carpeta build contiene la solución PracticaSTL.sln generada con CMake. Cada ejercicio es un proyecto independiente dentro de la solución.

1. Abrir build/PracticaSTL.sln con Visual Studio 2022.
2. En el Explorador de soluciones, seleccionar el proyecto del ejercicio.
3. Verificar que la configuración sea Release y la plataforma x64.
4. Presionar Ctrl+F5 para compilar y ejecutar sin depuración.
5. Para depurar, establecer puntos de interrupción y presionar F5.

Proyecto	Archivo fuente	Ejecutable
Ejercicio 1	ejercicio1_vector_ordenamiento.cpp	ejercicio1_vector_ordenamiento.exe
Ejercicio 2	ejercicio2_promedio_mayor.cpp	ejercicio2_promedio_mayor.exe
Ejercicio 3	ejercicio3_lista_tareas.cpp	ejercicio3_lista_tareas.exe
Reto extra	reto_extra_estadisticas.cpp	reto_extra_estadisticas.exe

Los ejecutables se generan en la carpeta build/Release.

También se puede abrir la carpeta 03_STL_Cpp como proyecto CMake en Visual Studio o Visual Studio Code. El archivo CMakePresets.json define la configuración visual-studio-2022 y la carpeta build.

COMANDOS CON CMAKE

```
cmake --preset visual-studio-2022
cmake --build build --config Release
```

COMPILACIÓN DIRECTA CON MSVC

```
cl /std:c++17 /EHsc /W4 /permissive- ejercicio1_vector_ordenamiento.cpp
ejercicio1_vector_ordenamiento.exe
```

7. Conclusiones

Los ejercicios muestran que la STL permite resolver problemas de almacenamiento, ordenamiento, búsqueda y agregación con poco código y alto rendimiento. vector es la opción natural para recorridos y acceso por índice; list conviene cuando se inserta o elimina en los extremos; y los algoritmos sort, accumulate, max_element, min_element y count_if evitan bucles manuales propensos a errores. La solución de Visual Studio y los archivos CMake permiten compilar y probar cada programa de forma reproducible.

Anexo A. Configuración CMake

La configuración declara los cuatro ejecutables, fija C++17 y activa advertencias de compilación para MSVC y para compiladores compatibles con GCC/Clang.

```
cmake_minimum_required(VERSION 3.20)
project(PracticaSTL LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(ejercicio1_vector_ordenamiento ejercicio1_vector_ordenamiento.cpp)
add_executable(ejercicio2_promedio_mayor ejercicio2_promedio_mayor.cpp)
add_executable(ejercicio3_lista_tareas ejercicio3_lista_tareas.cpp)
add_executable(reto_extra_estadisticas reto_extra_estadisticas.cpp)

if(MSVC)
    target_compile_options(ejercicio1_vector_ordenamiento PRIVATE /W4 /permissive-)
    target_compile_options(ejercicio2_promedio_mayor PRIVATE /W4 /permissive-)
    target_compile_options(ejercicio3_lista_tareas PRIVATE /W4 /permissive-)
    target_compile_options(reto_extra_estadisticas PRIVATE /W4 /permissive-)
else()
    target_compile_options(ejercicio1_vector_ordenamiento PRIVATE -Wall -Wextra -Wpedantic)
    target_compile_options(ejercicio2_promedio_mayor PRIVATE -Wall -Wextra -Wpedantic)
    target_compile_options(ejercicio3_lista_tareas PRIVATE -Wall -Wextra -Wpedantic)
    target_compile_options(reto_extra_estadisticas PRIVATE -Wall -Wextra -Wpedantic)
endif()
```

CMakePresets.json

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "visual-studio-2022",
      "displayName": "Visual Studio 2022 x64",
      "generator": "Visual Studio 17 2022",
      "architecture": "x64",
      "binaryDir": "${sourceDir}/build"
    }
  ],
  "buildPresets": [
    {
      "name": "release",
      "configurePreset": "visual-studio-2022",
      "configuration": "Release"
    }
  ]
}
```